

=====

VERIFYFLOW

=====

Multi-Org Document Approval Platform -- AWS Cloud Architecture Project Proposal (v2 -- Upgraded Scope)

Developer: Aravindan K

1. Scope & Rationale

=====

This project is scoped to what an AWS Certified Solutions Architect - Associate (SAA-C03) credential actually proves, plus a small set of adjacent AWS-native services that a competent associate-level architect would reasonably reach for without needing separate DevOps, ML, or security-specialist certification. The line is drawn deliberately: everything added in this v2 upgrade is still a design/configuration decision, not a programming-heavy automation one.

2. Problem Statement

=====

Organizations that need documents reviewed and formally approved or rejected -- vendor onboarding paperwork, compliance forms, contracts, cargo/customs-style submissions -- still largely run this process through email chains, shared folders, and spreadsheet trackers. Industry research on approval workflows converges on the same set of recurring failures:

1. No visibility into status, causing real time loss.

When approvals move through email or shared drives, organizations often have no reliable way to tell where a document is, who still needs to act on it, or how long it has been waiting -- so managers chase people and staff hunt for files instead of doing their jobs. Reporting suggests employees can lose roughly 15% of a workweek simply waiting on stuck approvals, and each manual approval cycle -- with its back-and-forth and follow-up -- has been estimated to cost an organization \$150-\$500 in lost productivity. At scale, manual approval processes have been found to inflate process costs by around 31% and stretch cycle times by more than 2x compared to automated alternatives.

2. No trustworthy audit trail.

Email-based approvals scatter the evidence of a decision across individual inboxes; spreadsheet trackers can be edited after the fact with no record of the change. When a regulator or auditor asks "who approved this, when, and on what basis," an "we always check this" answer is not accepted -- documented, unalterable evidence is required. Manual, undocumented approval processes have been associated with roughly 2-3x higher compliance risk, largely because required steps are easy to skip and hard to prove after the fact.

3. No reliable boundary between organizations sharing a platform.

Where multiple organizations submit and review documents on shared infrastructure, cross-tenant data exposure is a recognized, high-frequency failure mode -- industry analysis identifies broken access control (including cross-tenant access) as the most serious class of web application security risk, and a large share of SaaS platforms report insufficient visibility into whether their tenant isolation is actually holding. The consequence when it fails is not minor: average global data breach cost has been estimated at \$4.4-4.9 million, on top of regulatory exposure and every affected organization's own notification obligations.

4. No resilience guarantees.

If the review process depends on a single mail server, a single shared drive, or a single application instance, any outage stalls every organization's pending approvals simultaneously -- there is no isolation between "the system is down" and "every organization's work stops."

The problem, stated simply: there is no lightweight, secure, tenant-isolated, and reliably available system for multiple organizations to submit documents, route them through a defensible approval decision, and retain a trustworthy record of what happened.

3. Existing Systems in the Market

=====

This problem is not unsolved -- it's a mature commercial category. Being upfront about what already exists matters, because it sets honest expectations for what this project is and isn't.

+-----+-----+-----+-----+			
System	What it actually is	Approach	
+-----+-----+-----+-----+			
DocuWare	Cloud/on-prem document management + workflow platform, 20,000+ organizations, Gartner-rec.	AI-powered document capture (OCR/IDP), drag-and-drop workflow builder, ERP/SAP/ Microsoft integrations, tiered	
		per-user pricing (~\$20-70/mo)	
+-----+-----+-----+-----+			
Cflow	No-code BPM/workflow automation platform	Visual drag-and-drop form + rules builder, multi-level conditional approval routing,	
		starts around \$11/user/month	
+-----+-----+-----+-----+			
Moxo	Client-facing workflow and collaboration platform	Structured external client approval journeys, messaging + task assignment bundled	
+-----+-----+-----+-----+			
Nintex	Enterprise workflow auto,	Complex approval processes,	

	deep Microsoft/SharePoint	document generation, process	
	integration	analytics, enterprise pricing	
		(~\$900+/month)	
+-----+-----+-----+-----+			
Athento	Document lifecycle +	Queue-based review stages,	
	traceability platform	conditional lifecycle	
		transitions, OTP-based app.	
+-----+-----+-----+-----+			

The pattern across all of them: mature, no-code/low-code SaaS products, built by teams over years, sold on ease-of-use and breadth of integration (ERP, CRM, e-signature, OCR), priced per user per month, and hosted on infrastructure the buyer never sees or thinks about.

4. How VerifyFlow Differs From These Systems

=====

Stated plainly: VerifyFlow is not a competitor to DocuWare or Cflow, and shouldn't be pitched as one. It differs in kind, not just in maturity:

+-----+-----+-----+-----+			
Dimension	Existing Systems	VerifyFlow	
+-----+-----+-----+-----+			
What's sold	A finished product --	The infrastructure underneath	
	workflow logic, UI, etc.	a product -- AWS architecture	
+-----+-----+-----+-----+			
Build approach	No-code/low-code,	Hand-built app + hand-designed	
	configured by admins.	cloud architecture from scratch	
+-----+-----+-----+-----+			
Hosting	Abstracted away entirely.	Explicitly exposed VPC, RDS	
		replica, subnets, and IaC	
+-----+-----+-----+-----+			
Doc. Intel	Core feature (OCR/AI).	Explicitly out of scope.	
+-----+-----+-----+-----+			
Integrations	Extensive (40+).	None.	
+-----+-----+-----+-----+			
Multi-tenancy	Handled invisibly by SaaS	Enforced at the application	
	backend.	layer with cognito org_id	
+-----+-----+-----+-----+			
Audit trail	A product feature.	Implemented via versioned,	
		deny-delete S3 bucket policies	
+-----+-----+-----+-----+			
Resilience	Uptime SLAs.	Tested AZ failure, RDS	
		failover, CFN stack rebuild	
+-----+-----+-----+-----+			
Cost model	Per-user/month subscription	Pay-as-you-go, sized and	
	fee.	itemized line-by-line	

+-----+-----+-----+		
Purpose	Solve a business problem. Demonstrate AWS Well-Arch	
	pillars hands-on	
+-----+-----+-----+		

The honest takeaway:

Existing systems solve this problem better, faster, and more completely than VerifyFlow ever will -- they have OCR, e-signatures, ERP integrations, and years of production hardening that a portfolio project can't and shouldn't try to replicate. VerifyFlow's value isn't in out-competing them on features; it's in proving, with a working system and measured evidence, that the person building it can design and operate the kind of AWS infrastructure a real product like DocuWare or Cflow would actually run on. It's an infrastructure-and-operations demonstration wrapped around a minimal version of the same problem these products solve -- not an attempt to replace them.

If asked directly, "why not just use Cflow?" -- the honest answer is: "You should, if the goal is solving this problem in production. This project exists to prove I can build and run the AWS foundation underneath a system like that, not to replace it."

5. Solution

=====

VerifyFlow addresses this with a multi-org document approval platform built on AWS infrastructure, where the infrastructure -- codified, tiered, and measured -- is the actual point of the project.

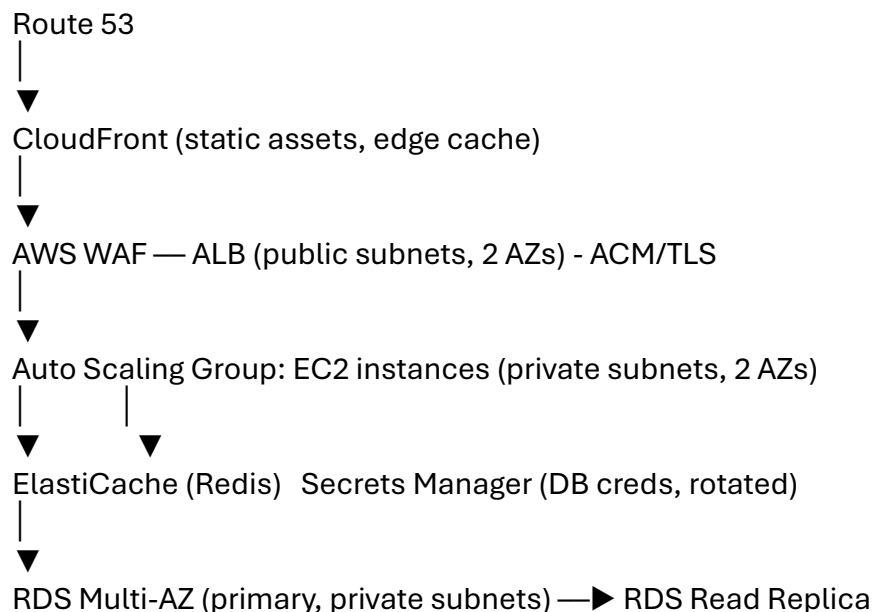
- Status tracking replaces email/spreadsheet tracking -- every document has one authoritative status (pending, approved, rejected, revision_requested) in a relational database, queryable at any time.
- Every decision is logged, immutably -- approvals and rejections write an append-only row to an audit table, with reviewer identity, org, reason, and timestamp -- so "who approved this and why" has a real, provable answer, protected from after-the-fact tampering.
- Organization boundaries are enforced at the application layer, on every request -- every query checks the requester's org_id against the resource being accessed, so one organization's reviewers cannot see or act on another's documents. Credentials used to reach the database are themselves rotated and never hardcoded.
- Availability is designed in and measured, not assumed -- compute runs across two Availability Zones behind a load balancer, the database runs Multi-AZ with a read replica, and the infrastructure itself is defined as code so it can be rebuilt identically after a disaster, not just recovered by hand.
- Performance is a first-class design input -- a caching tier absorbs repeated reads (e.g., "list my pending approvals"), and static assets are served from the edge, so response times stay low as load grows.

- The perimeter is defended, not just the network -- a managed web application firewall sits in front of the public entry point, filtering common attack patterns before they ever reach the application.
- Cost is a first-class design input -- storage tiering, right-sized instances, scheduled scaling, and a documented cost-per-organization estimate keep running cost proportional to actual usage, not guesswork.
- The infrastructure is reproducible -- the entire stack is defined in CloudFormation, so "how would you rebuild this if the account were lost" has a real answer: "aws cloudformation deploy", not a memory of console clicks.

The solution still deliberately does not attempt to solve document content problems (OCR, classification, automated compliance checking) -- those require ML/AI skills not being claimed here. What changed in this upgrade is depth of execution on the infrastructure and operations side: caching, perimeter defense, secret management, read scaling, and -- most importantly -- Infrastructure-as-Code, which was the single biggest credibility gap in the original scope.

6. Architecture

=====



- + S3 bucket -- document storage (SSE-KMS encryption, lifecycle rules)
- + S3 bucket -- audit log exports (versioned, deny-delete bucket policy)
- + Cognito User Pool -- authentication, org_id custom attribute
- + CloudWatch -- dashboards + alarms driving Auto Scaling and SNS alerts
- + IAM roles -- scoped per component (no shared/broad roles)
- + CloudFormation -- entire stack defined as code, one deployable template set

Compute

- EC2 Auto Scaling Group, min 2 / desired 2 / max 4, spread across 2 AZs (never zero capacity per AZ)
- Launch Template: Amazon Linux 2023, t3.medium (burstable, cost-appropriate for moderate traffic -- sizing decision documented against CloudWatch utilization data, not guessed)
- Target tracking scaling policy on CPU (~60%), plus a scheduled scaling action for predictable off-hours reduction
- No public IPs, no SSH -- access via Session Manager (SSM)
- ALB health checks hit a real /health endpoint that verifies DB and cache connectivity, not just process liveness

Networking & Security

- VPC with public subnets (ALB only) and private subnets (EC2, RDS, ElastiCache)
- AWS WAF attached to the ALB -- managed rule groups (SQL injection, common exploits) plus a rate-based rule to blunt basic abuse, before traffic ever reaches EC2
- Security Groups: ALB -> EC2 on app port only; EC2 -> RDS on DB port only; EC2 -> ElastiCache on cache port only
- IAM roles scoped per function (EC2 instance role != audit-export role != any other component role)
- Secrets Manager holds the RDS credentials with automatic rotation -- the application retrieves them at runtime via IAM role, nothing hardcoded or stored in environment files
- S3 buckets: SSE-KMS encryption, public access blocked at bucket and account level
- Audit log bucket: versioning on, bucket policy denies delete -- protects log integrity

Data

- RDS Multi-AZ primary (Postgres or MySQL), automated backups
- RDS Read Replica -- read-heavy queries (e.g., dashboard/status listings) are routed here, keeping load off the primary and directly demonstrating the "performance optimization: database options" domain
- ElastiCache (Redis) -- caches frequent, low-volatility reads (pending-approval counts, org metadata), with a short TTL and explicit cache-invalidation on write, so the trade-off between freshness and speed is a documented, deliberate decision
- Tables: organizations, documents, reviews, audit_log
- Every table carries org_id -- isolation enforced at the application layer (WHERE org_id = current_user.org_id on every query)

Application Logic

1. Org member uploads a document (presigned S3 URL -- backend never touches raw file bytes)
2. Metadata row written to documents table, status = pending; cache entry for that org's pending count invalidated

3. Reviewer within the same org approves, rejects, or requests revision, with a reason code + free text
4. Every state change writes one row to audit_log (who, what, when, which org)

Authentication

- Cognito User Pool, org_id stored as a custom attribute, checked in application code on every request -- the tenant-isolation mechanism, kept at the application layer deliberately (not an IAM condition-tag policy), so it stays fully explainable and testable end-to-end

Cost Optimization

- S3 lifecycle rule: documents older than 90 days -> S3-IA
- ASG scheduled scale-down during off-hours
- ElastiCache reduces RDS read load, which allows a smaller primary instance class than would otherwise be needed -- a direct, quantifiable cost trade-off to document
- Instance type choices documented with reasoning, backed by actual CloudWatch utilization numbers once running
- AWS Budgets + SNS alert configured for a monthly spend threshold

Infrastructure as Code

- The full stack -- VPC, subnets, security groups, ALB, ASG/Launch Template, RDS (primary + replica), ElastiCache, S3 buckets, IAM roles, WAF, Secrets Manager -- defined in CloudFormation templates, parameterized so the same template deploys a dev and a prod stack
- This directly closes the previous version's biggest credibility gap: the stack can be destroyed and rebuilt identically, on demand, which is provable in a live demo

7. Resilience Demonstration (Strongest Part of the Project)

=====

Concrete, measured proof beats a diagram. Set target RTO/RPO figures up front (e.g., RTO < 5 minutes, RPO < 5 minutes for RDS automated backups), then run and document three tests against them:

1. EC2 failure test -- manually terminate one instance. Observe ASG launch a replacement and ALB route around the gap. Record recovery time against the RTO target.
2. RDS failover test -- trigger "Reboot with failover" on the Multi-AZ instance via console. Record the downtime blip and full recovery time against target.
3. Full stack rebuild test -- tear down the CloudFormation stack entirely and redeploy it from the template in a clean AWS account or region. Record how long it takes to go from nothing to a healthy, passing-health-check system. This is the test that proves the IaC investment is real, not decorative.

These three measured numbers are more valuable in an interview than any

architecture diagram -- they prove the resilience domain was actually exercised against a stated target, not just designed on paper.

8. Unbiased Evaluation (Updated for v2)

=====

What improved from v1

- The IaC gap is closed. CloudFormation templates mean the stack is reproducible and provable -- the single highest-leverage fix identified in the original evaluation.
- Two more Well-Architected pillars now have concrete evidence, not just design intent: performance (caching + read replica, ideally with a measured before/after latency number) and security (WAF + Secrets Manager add real, demonstrable controls at the perimeter and credential layer).
- The resilience story is stronger -- a full stack teardown-and-rebuild test is a genuinely rare thing to see in a portfolio project and is a strong, concrete talking point.

What's still true, stated plainly

- The application logic is still simple. Adding caching and a read replica makes the infrastructure more sophisticated, but approve/reject/revision-request is still not a hard software engineering problem. This project proves infrastructure and operations competency, not algorithmic or product depth -- that framing hasn't changed.
- It is still an infrastructure showcase wearing a product costume, just a more capable one. Don't let the added services shift the pitch from "I can build a resilient, cost-aware, secure AWS deployment" into "I built a real product" -- the second claim still isn't supported.
- The service list is more distinctive than v1, but still associate-tier. WAF, ElastiCache, Secrets Manager, and CloudFormation are common next steps for SAA-C03 holders, not advanced or rare -- this raises the floor, it doesn't make the project unique.
- CI/CD is still absent. Deploying the CloudFormation stack is presumably still a manual "aws cloudformation deploy" command run by hand, not triggered by a pipeline. That's fine and consistent with the stated scope, but if asked "how does this get deployed on every code change," the honest answer is still "it doesn't, yet" -- worth having that answer ready rather than being caught by the question.

9. Recommended Use

=====

Good fit for:

- A stronger cloud-focused portfolio piece than a bare associate-level deployment -- direct, hands-on supporting evidence for the SAA-C03 credential plus early IaC exposure
- Entry-level to early-mid cloud/infrastructure roles, cloud support roles,

or infra-adjacent software roles where reproducible deployments matter

- A 2-3 week build -- larger than the v1 scope, but still bounded; resist the temptation to keep adding services

Not a good fit for:

- Positioning as a flagship product portfolio piece -- it's still an infrastructure exercise, not a product
- Roles that specifically weight software engineering depth (data modeling, algorithms, business logic complexity) over infrastructure/ops skill
- Claiming CI/CD or container orchestration experience -- that's still not part of this build

Honest presentation language:

> "Built and deployed a multi-org approval workflow on AWS, defined entirely in CloudFormation -- VPC segmentation, Multi-AZ RDS with a read replica, ElastiCache for hot-path reads, WAF at the perimeter, and Secrets Manager for credential rotation. Measured failover recovery time and validated a full stack teardown-and-rebuild from the template."

This is accurate, provable under questioning, and reflects genuine added depth without overclaiming DevOps or ML scope the credential doesn't support.

10. Realistic Build Timeline

=====

+-----+-----+-----+-----+	
Week	Focus Deliverables
+-----+-----+-----+-----+	
1	CloudFormation skeleton VPC/subnets, security groups, IAM roles -- infra as code from day one, not retrofitted
+-----+-----+-----+-----+	
2	Compute & Database Tiers EC2 + ASG + ALB + WAF, RDS Multi-AZ primary + read replica, Secrets Manager integration
+-----+-----+-----+-----+	
3	Logic & Caching Tiers App logic (upload, review, approve/reject), Cognito auth, org_id, ElastiCache Redis
+-----+-----+-----+-----+	
4	Audit & Monitoring Tiers Audit logging, S3 lifecycles, CloudFront static CDN, CloudWatch dashboards & alerts
+-----+-----+-----+-----+	
5	Resilience Drills Terminate EC2 instance, RDS failover, full stack rebuild, budget setup and cost review
+-----+-----+-----+-----+	

11. Natural Next Step (if pursued later)

=====

With IaC, caching, perimeter defense, and read scaling now in place, the next credible growth step is CI/CD -- a simple pipeline (e.g., CodePipeline/CodeBuild, or GitHub Actions deploying the CloudFormation stack) that deploys on every merge to main. That is the one item still explicitly out of scope, and it's the natural, honest "what's next" answer in an interview: "the stack is fully codified -- automating its deployment is the next step I'd take."

12. Sources (Problem Statement & Market Comparison Research)

=====

- KDI Office Technology -- Why Approval Processes Slow Organizations Down (<https://www.kdi-inc.com/document-management/why-approval-processes-slow-organizations-down/>)
- InfoSeeMedia -- Why Manual Approval Workflows Fail in 2026 (<https://infoseemedia.com/business/saas/why-manual-approval-workflows-fail/>)
- Cflow / Seyarc AI -- Compliance Approval Workflow: Steps to Build Audit-Ready Controls (<https://www.cflowapps.com/compliance-approval/workflow/>)
- Moxo -- What is a Document Approval Workflow? (<https://www.moxo.com/blog/document-approval-workflow-guide>)
- Bugstrix -- Multi-Tenant SaaS Security Testing: How to Prevent Cross-Tenant Data Leaks (<https://bugstrix.com/blogs/multi-tenant-saas-security-testing-how-to-prevent-cross-tenant-data-leaks/>)
- FluxForce -- AI Multi-Tenant Security (<https://www.fluxforce.ai/superhumans/ai-cloud-security-architect>)
- Clerk -- What Are the Risks and Challenges of Multi-Tenancy? (<https://clerk.com/blog/what-are-the-risks-and-challenges-of-multi-tenancy>)
- Qrvey -- 2026 Tenant Isolation Guide (<https://qrvey.com/blog/tenant-isolation/>)
- SuiteFiles -- Top 6 Document Approval Workflow Software for 2026 (<https://www.suitefiles.com/document-approval-workflow-software/>)
- Cflow -- Top 15 Document Approval Software to Consider for 2026 (<https://www.cflowapps.com/document-approval-software-system/>)
- Cflow -- Cflow vs Moxo: Workflow Automation Comparison (<https://www.cflowapps.com/compare/cflow-vs-moxo/>)
- Capterra -- DocuWare Software Pricing, Alternatives & More 2026 (<https://www.capterra.com/p/220466/DocuWare/>)
- CheckThat.ai -- DocuWare Pricing 2026: Plans, Costs & TCO Guide (<https://checkthat.ai/brands/docuware/pricing>)